

Day 1 Track Map — F1 → F2 → F3

By EOD: environment ready · first agent running · prompt + tool-call verified.

EXIT CRITERIA · DAY 1

✓

Repo cloned, venv active, deps installed

✓

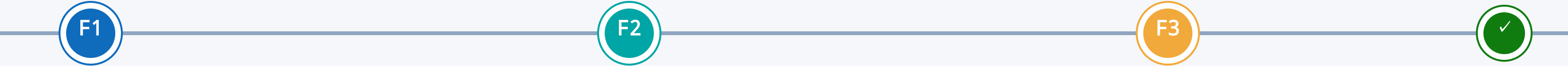
Foundry / OpenAI credentials verified

✓

Starter agent returns tool-grounded reply

✓

MCP client reachable from local runtime



MODULE F1 · 90 MIN

Agentic AI Primer

CONCEPT · TAXONOMY

•

Chatbot vs RAG vs Agent — decision tree

•

Agent anatomy: Reasoning · Tools · Memory

•

The ReAct-style reasoning loop

•

When an agent is the wrong tool

OUTPUT → shared mental model

MODULE F2 · 60 MIN

Microsoft Foundry Overview

PLATFORM · CAPABILITIES

•

Foundry capability stack (model → agent)

•

Model catalog, tools, tracing, evals

•

Position in Azure enterprise stack

•

Identity, network, governance surface

OUTPUT → platform map + naming

MODULE F3 · 120 MIN · HANDS-ON

Environment & Repo Setup

TOOLING · FIRST AGENT

•

Clone repo · Python venv · Azure login

•

Install LangGraph / Agent SDK / MCP SDK

•

Wire keys, endpoints, model deployments

•

Run starter agent: prompt → tool → response

OUTPUT → working local agent

Chatbot · RAG · Agent — Block-Level Contrast

Same input surface, three fundamentally different runtimes. Pick the simplest one that answers the requirement.

PATTERN 01 · STATELESS

Chatbot

USER → LLM → USER

```
graph LR; User[User] <--> LLM[LLM completion];
```

CHARACTERISTICS

- Single-turn or naive multi-turn
- Answer bounded by training corpus
- No external state, no side effects
- Deterministic latency (1 model call)

Use when: knowledge is static, task fits one prompt, no ground-truth freshness required.

FAIL MODE: hallucinated facts, stale data

PATTERN 02 · RETRIEVAL-AUGMENTED

RAG

USER → RETRIEVE → LLM → USER

```
graph LR; User[User] --> Retriever[Retriever top-k · embed]; Retriever --> LLM[LLM grounded]; LLM --> Answer[Answer]; VectorStore[VECTOR STORE] -.-> Retriever;
```

CHARACTERISTICS

- Deterministic pipeline (2 stages)
- Answers grounded in source docs
- No decisions, no side effects
- Read-only over an indexed corpus

Use when: Q&A over private/fresh docs; answers must cite sources.

FAIL MODE: retrieval miss, chunk fragmentation

PATTERN 03 · GOAL-DIRECTED

Agent

```
graph LR; subgraph ReasoningLoop [REASONING LOOP]; Plan[Plan] --> Act[Act]; Act --> Observe[Observe]; Observe --> Plan; end; Memory[MEMORY] -.-> Observe; Goal[GOAL/TOOLS] --> Plan; Plan --> Act; Act --> Observe; Observe --> Result[RESULT];
```

CHARACTERISTICS

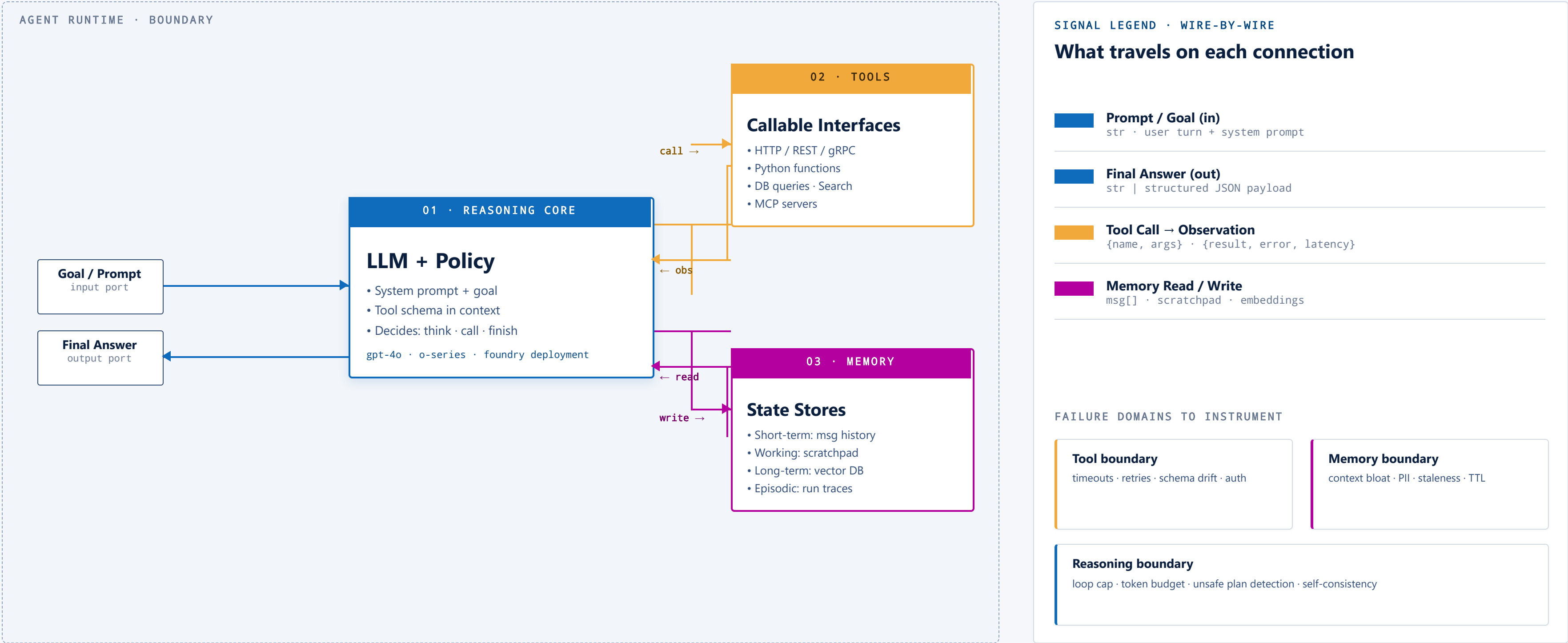
- Non-deterministic N-step trajectory
- Calls tools with real side effects
- Holds working + long-term memory
- Latency + cost bounded by loop cap

Use when: task needs decisions, multi-step execution, or actions in external systems.

FAIL MODE: loop divergence, unsafe tool calls

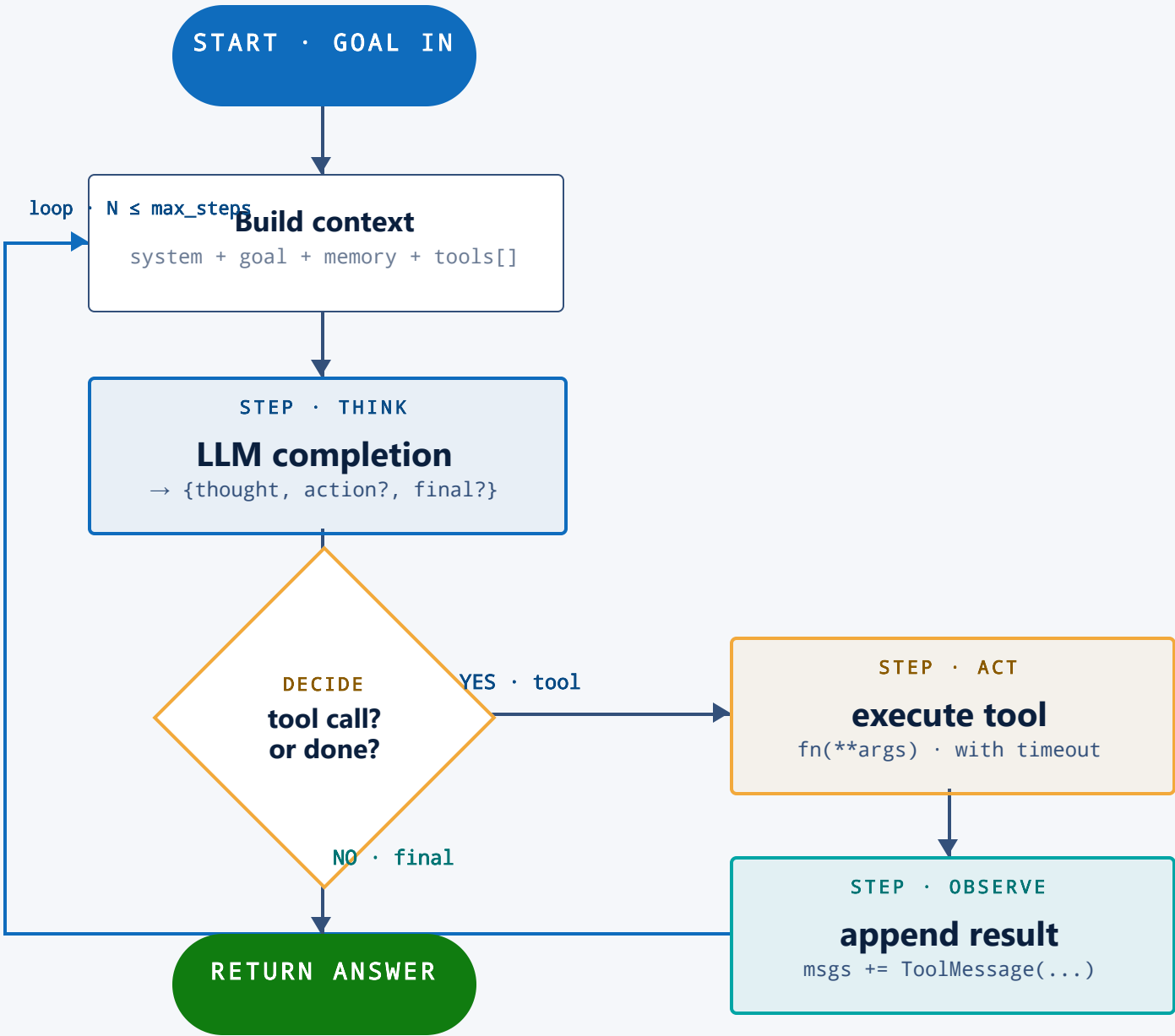
Agent Anatomy — Reasoning · Tools · Memory

A block diagram of what actually sits inside `agent.invoke()`. Every arrow is a real interface you can trace.



The Agent Reasoning Loop — Flowchart

ReAct-style trajectory: Think → Act → Observe, terminated by a stop condition. Guard rails on every arrow.



GUARD-RAILS

- **max_steps** · hard loop cap
- **token_budget** · per-run cap
- **allow_list** · tools whitelist
- **output_schema** · validate final
- **hitl** · human-in-the-loop gate

EXAMPLE TRAJECTORY

"What's the weather in Seattle right now?"

```
# t=0 · THINK
thought: user wants live weather → need get_weather tool
action : get_weather("Seattle, WA")

# t=1 · ACT + OBSERVE
observation: {"temp_f": 58, "cond": "cloudy",
              "wind_mph": 7, "ts": "2026-..." }

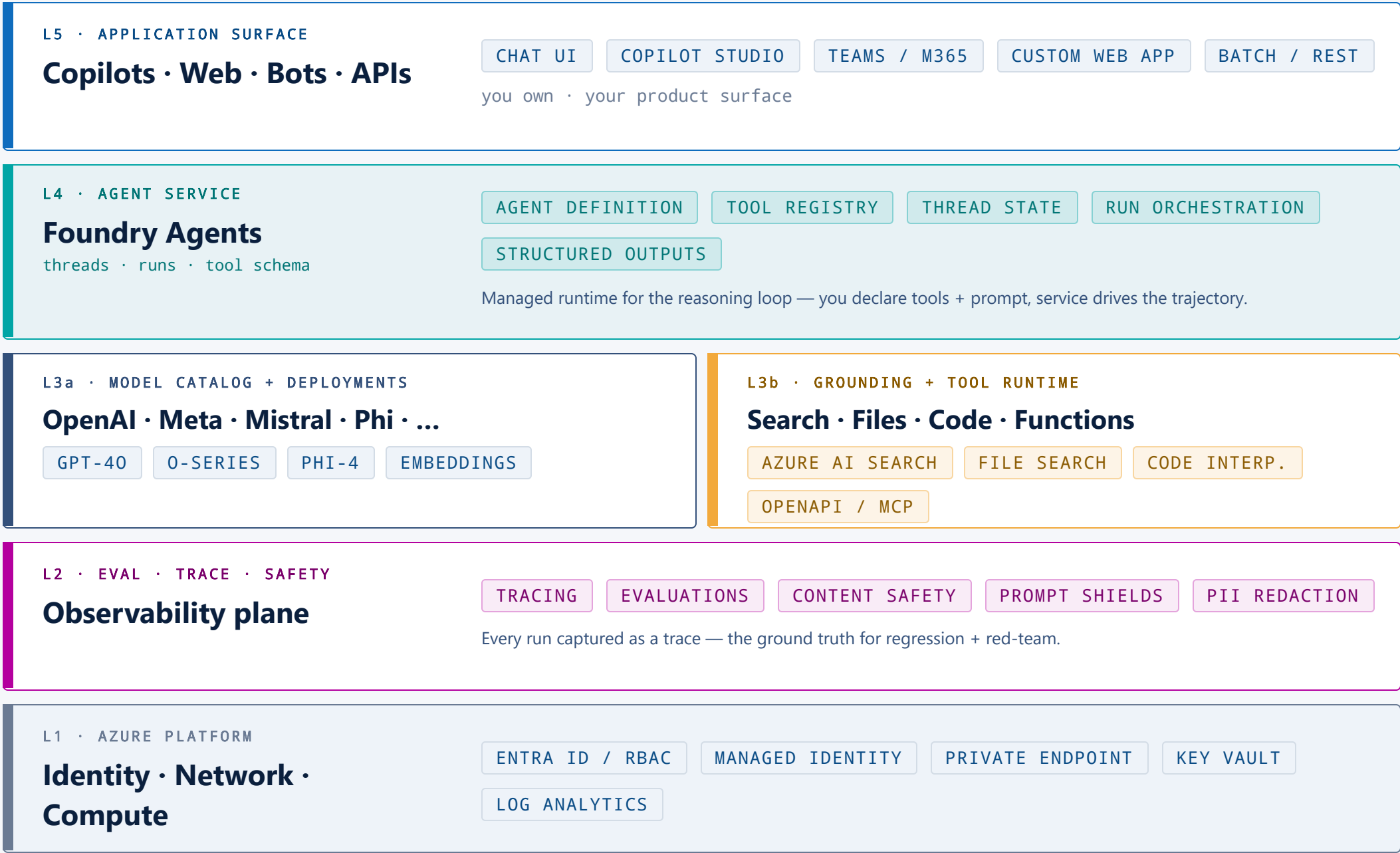
# t=2 · THINK
thought: have live data → can compose answer
action : FINAL

# t=3 · RETURN
"Seattle is 58 °F and cloudy, wind 7 mph."

# invariants
steps_used = 2           // ≤ max_steps (=6)
tools_used = ["get_weather"] // ⊂ allow_list
tokens_in  = 812         // ≤ token_budget
```

Microsoft Foundry — Capability Stack

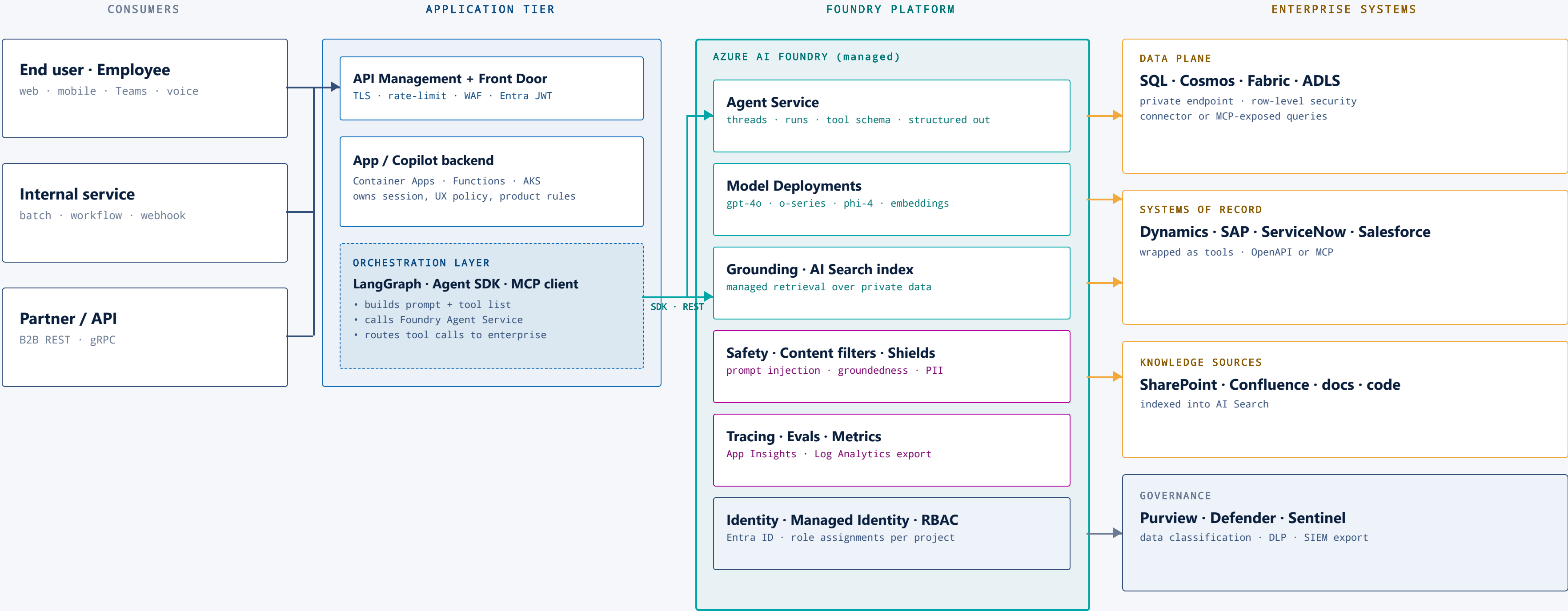
A layered block diagram of what Foundry gives you out of the box vs. what you still own. Read bottom-up: `infra` → `models` → `agent` → `app`.



OWNERSHIP MATRIX		
Foundry-managed vs. you-managed		
LAYER	FOUNDRY	YOU
Model hosting	✓	—
Agent runtime (loop, threads)	✓	—
Tool implementation	partial	✓
Prompt / policy	—	✓
Grounding data (docs, DB)	index only	✓
Traces + evals	✓	✓
Content safety filters	✓	tune
Product surface / UX	—	✓

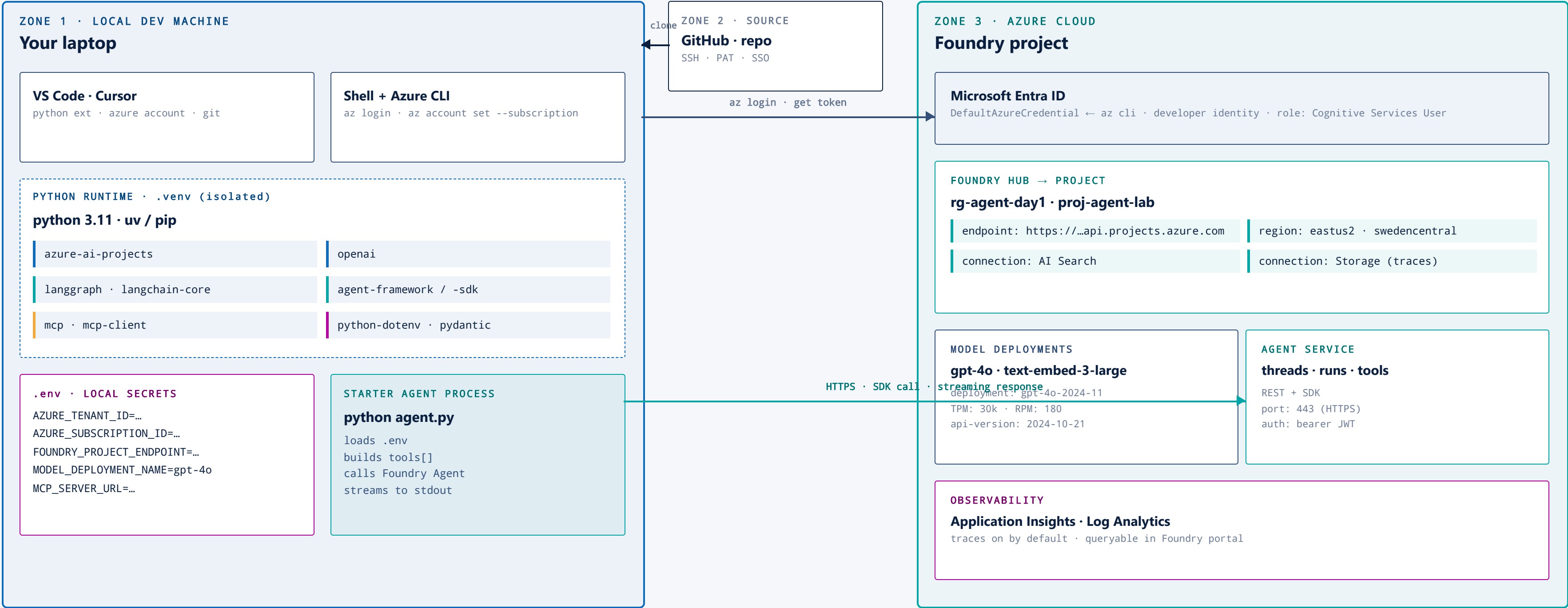
Foundry in the Enterprise Stack — System Wiring

Where Foundry sits between your consumers and your enterprise systems. Every arrow crosses a trust boundary — auth + logging on both sides.



Dev Environment — Wiring Diagram

Your laptop → GitHub → Azure. Every credential, every endpoint, every port that has to be reachable before slide 10.



Repo Clone → Deps → Auth → Smoke Test

6-step linear flowchart. Each step has a command + a verify assertion. Do NOT skip verify — later steps assume it passed.

1

Clone repo & enter directory

~15 SEC

\$ git clone git@github.com:acme/agent-day1.git && cd agent-day1

✓ VERIFY

\$ ls -l | head -n 10 → agent.py pyproject.toml .env.sample README.md

2

Create + activate Python venv

~5 SEC

\$ python3.11 -m venv .venv && source .venv/bin/activate

✓ VERIFY

\$ which python → /.../agent-day1/.venv/bin/python

3

Install SDKs — Foundry · LangGraph · Agent SDK · MCP

~60 SEC

\$ pip install -U azure-ai-projects openai \
langgraph langchain-core agent-framework mcp python-dotenv

✓ VERIFY

\$ python -c "import azure.ai.projects, langgraph, mcp; print('ok')"

→ ok

4

Authenticate to Azure & wire .env

~20 SEC

\$ az login --tenant \$TENANT_ID && cp .env.sample .env
\$ \$EDITOR .env # fill FOUNDRY_PROJECT_ENDPOINT + MODEL_DEPLOYMENT_NAME

✓ VERIFY

\$ az account show --query name -o tsv
\$ az account get-access-token --resource https://ai.azure.com -o none # silent ok

5

Ping model deployment · smoke test

~3 SEC

\$ python scripts/ping_model.py # calls chat.completions on \$MODEL_DEPLOYMENT_NAME

✓ VERIFY

→ "pong: gpt-4o-2024-11-20 · 214 tokens · 1.1s"

6

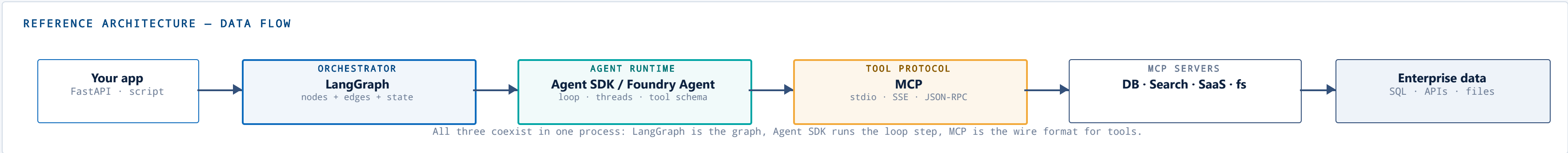
Run starter agent (next slide breaks down the flow)

READY

\$ python agent.py "What's the weather in Seattle right now?"

LangGraph · Agent SDK · MCP — Where Each Fits

Three different layers, not three competitors. Pick the orchestrator, pick the runtime, pick the tool protocol — independently.



ORCHESTRATOR

LangGraph

STATEFUL GRAPH · CONTROL FLOW
WHAT IT IS

A graph runtime: you declare nodes (functions) and edges (transitions) that share typed state. The reasoning loop becomes an explicit DAG.

```
# graph = plan → act → observe → decide
from langgraph.graph import StateGraph
g = StateGraph(AgentState)
g.add_node("think", think_fn)
g.add_node("tool", tool_fn)
g.add_conditional_edges("think", router)
```

Pick when: multi-agent · branching · retries · human-in-the-loop · you want full control of state transitions.

RUNTIME

Agent SDK / Foundry Agents

MANAGED LOOP · THREADS · RUNS
WHAT IT IS

A hosted agent primitive: define instructions + tools, service runs the think-act-observe loop, streams events back. You don't own the loop.

```
from azure.ai.projects import AIProjectClient
client = AIProjectClient(endpoint, cred)
agent = client.agents.create_agent(
    model="gpt-4o",
    instructions="You are...",
    tools=[get_weather_tool])
```

Pick when: straight tool-calling agent · want managed tracing/safety · don't need custom control flow.

TOOL PROTOCOL

MCP

MODEL CONTEXT PROTOCOL · JSON-RPC
WHAT IT IS

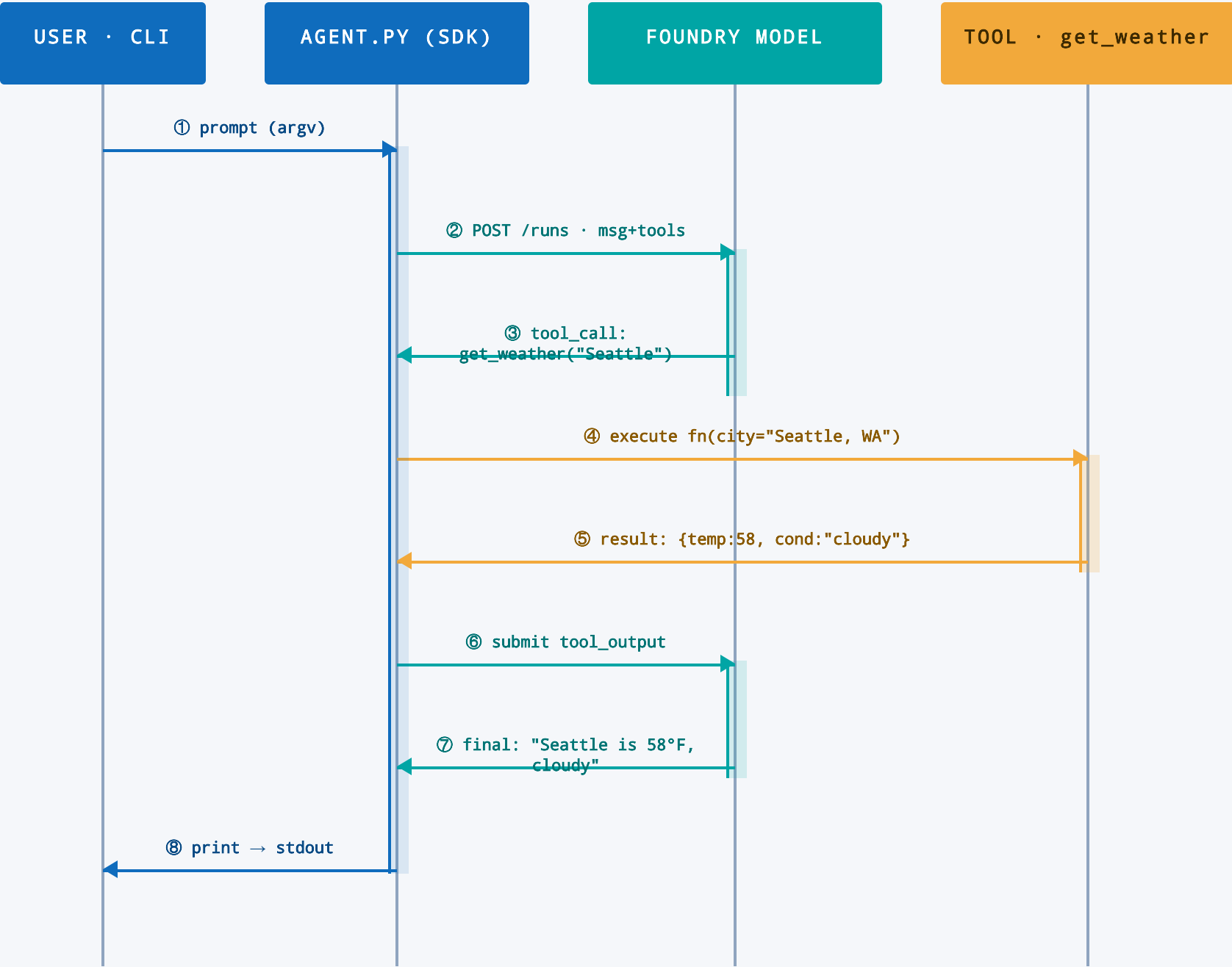
Open protocol for exposing tools + resources to any agent runtime. Server publishes a tool catalog; clients discover, invoke, stream results.

```
# any agent · any server, same wire
async with stdio_client(params) as (r, w):
    async with ClientSession(r, w) as s:
        await s.initialize()
        tools = await s.list_tools()
        await s.call_tool("get_weather", args)
```

Pick when: tools should be reusable across agents · integrating third-party tool servers · decoupling tool auth from agent code.

Starter Agent — Prompt · Tool Call · Response

The end-to-end wire trace for one python agent.py invocation. Numbers on the arrows == steps in the sequence diagram.



agent.py · MINIMAL WORKING SAMPLE

Prompt + one tool + one call

```
import os, sys; from dotenv import load_dotenv
from azure.identity import DefaultAzureCredential
from azure.ai.projects import AIProjectClient

load_dotenv()
client = AIProjectClient(
    endpoint = os.environ["FOUNDRY_PROJECT_ENDPOINT"],
    credential = DefaultAzureCredential() # ← az login

# — 1. define one tool —
def get_weather(city: str) -> dict:
    return {"temp_f": 58, "cond": "cloudy"}

TOOL_SCHEMA = [{"type": "function", "function": {
    "name": "get_weather",
    "parameters": {"type": "object",
        "properties": {"city": {"type": "string"}},
        "required": ["city"]}}}]

# — 2. create agent + thread —
agent = client.agents.create_agent(
    model = os.environ["MODEL_DEPLOYMENT_NAME"],
    instructions = "You are a weather assistant.",
    tools = TOOL_SCHEMA)
thread = client.agents.threads.create()

# — 3. run the reasoning loop —
client.agents.messages.create(thread.id,
    role="user", content=sys.argv[1])
client.agents.runs.create_and_process(
    thread.id, agent.id,
    tool_functions={"get_weather": get_weather})

# — 4. print final message —
for m in client.agents.messages.list(thread.id, order="asc"):
    if m.role == "assistant": print(m.content[0].text.value)
```